

Match Sale Countdown – Implementación de Contador dinámico en landing existente.

Contexto

En esta tarea se trabajó sobre una landing page existente para una campaña de “Match Sale”.

El componente de contador regresivo ya estaba implementado visualmente, pero presentaba una limitación importante:

- Los valores estaban **hardcodeados en HTML**, por lo que no se actualizaban automáticamente.

Esto generaba una mala experiencia de usuario y requería actualización manual constante.

Objetivo

Transformar un contador estático en un **contador dinámico en tiempo real**, manteniendo el diseño existente e integrándolo sin romper la estructura de la landing.

Problema detectado

El HTML original contenía valores fijos:

```
<div class="countdown">
  <div>02 <small>Días</small></div>
  <div>14 <small>Hs</small></div>
  <div>35 <small>Min</small></div>
</div>
```

Limitaciones:

- No reflejaba el tiempo real
- Requería edición manual

- No escalable para múltiples eventos
- Riesgo de información desactualizada

Solución implementada

Se reemplazaron los valores hardcoded por un sistema dinámico utilizando JavaScript.

1. Refactor del HTML

Se agregaron identificadores para manipular el contenido:

```
<div class="countdown">
  <div><span id="dias">00</span><small>Días</small></div>
  <span class="separador">:</span>
  <div><span id="horas">00</span><small>Hs</small></div>
  <span class="separador">:</span>
  <div><span id="minutos">00</span><small>Min</small></div>
  <span class="separador">:</span>
  <div><span id="segundos">00</span><small>Seg</small></div>
</div>
```

Se modificó el contenido HTML, para poder interactuar mas facilmente con Java Script y actualizar los valores numéricos sin afectar el contenido:

- Se agregó un ID para que Java Script pueda encontrarlo y modificarlo facilmente.
- Se agregó una etiqueta para separar el texto (dias) del número dinámico sin romper el diseño.

2. Implementación de lógica dinámica

Se desarrolló una función que:

- Obtiene la fecha actual
- Calcula la diferencia con la fecha del evento
- Convierte esa diferencia en días, horas, minutos y segundos
- Actualiza el contenido del HTML cada segundo

```
const fechaEvento = new Date("2026-05-01T18:00:00");

function actualizarContador() {
  const ahora = new Date();
  const diferencia = fechaEvento - ahora;

  if (diferencia <= 0) return;

  const dias = Math.floor(diferencia / (1000 * 60 * 60 * 24));
  const horas = Math.floor((diferencia / (1000 * 60 * 60)) % 24);
```

```
const minutos = Math.floor((diferencia / (1000 * 60)) % 60);
const segundos = Math.floor((diferencia / 1000) % 60);

document.getElementById("dias").textContent = String(dias).padStart(2,
"0");
document.getElementById("horas").textContent =
String(horas).padStart(2, "0");
document.getElementById("minutos").textContent =
String(minutos).padStart(2, "0");
document.getElementById("segundos").textContent =
String(segundos).padStart(2, "0");
}

setInterval(actualizarContador, 1000);
actualizarContador();
```

Mejora visual

Se realizaron mejoras visuales para:

- Incorporar separadores (:)
- Mejorar alineación con Flexbox
- Mantener consistencia con el diseño existente

Nota: el código HTML recibido, contenía dentro el código CSS. Se separó en una pestaña específica de CSS, siguiendo las buenas prácticas de código.

Resultado

- ✓ Contador actualizado en tiempo real
- ✓ Eliminación de mantenimiento manual
- ✓ Mejora en la experiencia de usuario
- ✓ Componente reutilizable para futuros eventos

Aprendizajes

- Manipulación del DOM con JavaScript
- Manejo de fechas y tiempos
- Refactor de componentes existentes sin romper diseño
- Importancia de evitar hardcoding en UI dinámica

Ahora

Se recibe un archivo HTML con un contador hardcodeado y un bloque `<style>` interno.

s